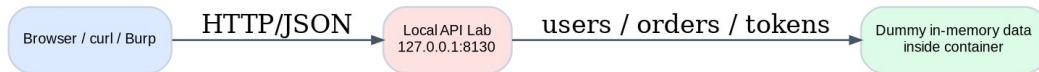


RED TEAM LEARNING PATH

MODULE 13

API SECURITY

REST/JSON attack surface, authorization, business logic, inventory & GraphQL



Isolated local API training architecture

Authorized-lab only

Semua request aktif dalam module ini hanya untuk localhost/container milik sendiri atau deliberately vulnerable training platform seperti PortSwigger Web Security Academy. Menemukan endpoint API publik tidak berarti endpoint tersebut boleh diuji.

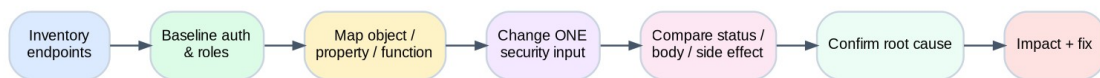
Field	Value
Primary OS	Omarchy / Arch Linux
Prerequisite	Modules 00-12
Core tools	Burp Suite, curl, jq, Docker, Python
Local target	127.0.0.1:8130
Outcome	Mampu menilai API secara sistematis dari inventory sampai finding + remediation

1. Learning Objectives

- [] Memahami perbedaan authentication, object-level, property-level, dan function-level authorization.
- [] Membangun inventory endpoint dari traffic dan OpenAPI documentation.
- [] Menguji BOLA/IDOR tanpa menganggap UUID sebagai kontrol akses.
- [] Menguji broken object property-level authorization / mass assignment.
- [] Menguji broken function-level authorization pada operation administratif.
- [] Menilai rate/resource limits dan sensitive business flows tanpa stress testing.
- [] Memahami API versioning, deprecated/shadow endpoints, dan inventory management.
- [] Menilai JWT/API token secara kontekstual tanpa menganggap format token sebagai trust.
- [] Memahami GraphQL endpoint/schema/authorization methodology.
- [] Menulis finding yang menyertakan request, response delta, root cause, impact, dan remediation.

2. API Security Mental Model

API security bukan sekadar "web pentesting tanpa HTML". API biasanya mengekspos object identifiers, structured JSON, role-dependent operations, machine-readable documentation, mobile/SPA backends, dan business flows yang mudah diautomasi. Karena itu tester perlu memetakan data model dan authorization model bersamaan.



API testing loop: inventory dulu, baru mutation terkontrol

Think like a tester

Untuk setiap endpoint, tanyakan: siapa caller-nya, object apa yang disentuh, property mana yang boleh dibaca/diubah, function apa yang dipanggil, resource apa yang dikonsumsi, dan business invariant apa yang harus tetap benar?

3. OWASP API Security Top 10 2023

Risk	Inti
API1 BOLA	Caller boleh endpoint-nya, tetapi tidak boleh object yang dipilihnya.
API2 Broken Authentication	Identity/token handling gagal sehingga caller dapat mengambil identitas lain.
API3 BOPLA	Property sensitif terbaca atau bisa diubah tanpa authorization yang benar.
API4 Resource Consumption	Request dapat menghabiskan CPU/memori/bandwidth/biaya tanpa guardrail.
API5 BFLA	User dapat memanggil function/operation yang seharusnya hanya untuk role tertentu.
API6 Sensitive Business Flows	Flow valid disalahgunakan secara otomatis/berlebihan sehingga merugikan bisnis.
API7 SSRF	API mengambil URI user tanpa validasi destination yang benar.
API8 Misconfiguration	Debug, CORS, TLS, headers, default credentials, verbose errors, dll.
API9 Inventory Management	Versi lama, shadow endpoint, host/API docs tidak terinventarisasi.
API10 Unsafe Consumption	Data/response third-party API dipercaya terlalu tinggi.

OWASP 2023 menggabungkan Excessive Data Exposure dan Mass Assignment ke API3 karena akar masalahnya sama: authorization pada property object tidak ditegakkan dengan benar.

4. Tooling on Omarchy

Module sebelumnya sudah memasang Burp Suite. Untuk API workflow kita cukup memastikan Docker, curl, jq, dan Python tersedia.

Verify tooling

```
omarchy pkg add jq python

curl --version
jq --version
python --version
sudo docker version
```

Optional clients

Bruno/Postman boleh dipakai untuk productivity, tetapi Burp + curl tetap penting karena keduanya membuat raw HTTP behavior terlihat jelas. Jangan menyimpan production token/API key ke collection yang akan di-commit.

5. Local API Lab

Lab menggunakan dummy in-memory data dan deliberately broken authorization. Tidak ada external callback, credential nyata, atau persistent database. Container dipublish hanya ke localhost.

Start local training API

```
cd modules/13-api-security/labs

./workspace-init.sh
./api-lab.sh build
./api-lab.sh create

curl -s http://127.0.0.1:8130/ | jq .
```

Containment

Container berjalan non-root, cap-drop ALL, no-new-privileges, read-only filesystem, tmpfs terbatas, dan tidak mempunyai host volume mount. Vulnerability sengaja berada pada application authorization/business logic, bukan host escape.

Hands-on 5.1 - Establish two identities

```
curl -s -X POST http://127.0.0.1:8130/api/v1/login \
-H 'Content-Type: application/json' \
-d '{"username":"alice","password":"training123"}' | jq .

curl -s -X POST http://127.0.0.1:8130/api/v1/login \
-H 'Content-Type: application/json' \
-d '{"username":"bob","password":"training123"}' | jq .
```

Simpan hanya training token di notes lokal. Kita membutuhkan minimal dua identity supaya authorization test bukan sekadar menebak status code.

6. API Recon & Documentation Discovery

API recon dimulai dari traffic yang dipakai aplikasi, route patterns, HTTP methods, content types, error messages, JavaScript/mobile references, documentation, dan machine-readable schema seperti OpenAPI. Dokumentasi adalah map, bukan authorization grant.

Hands-on 6.1 - OpenAPI inventory

```
curl -s http://127.0.0.1:8130/openapi.json \
-o module13-api-security/01-recon/openapi.json

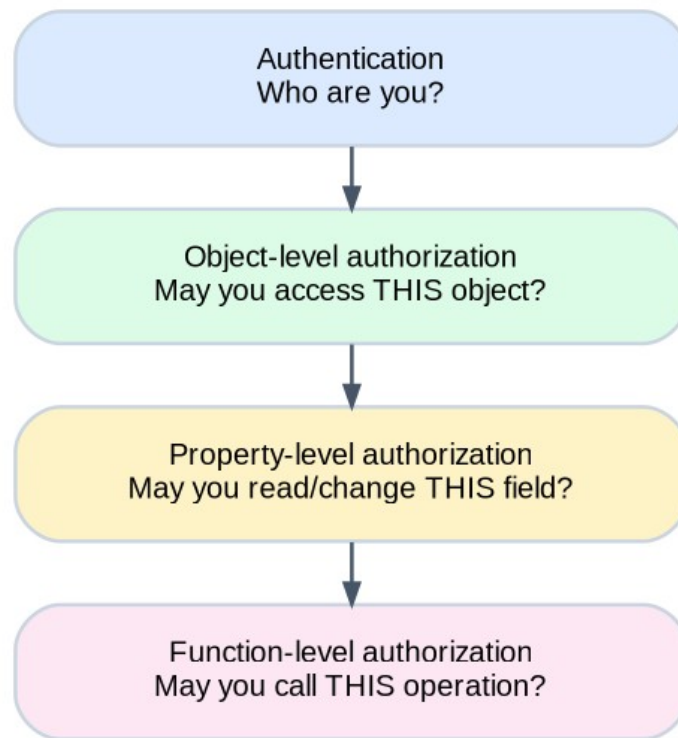
python labs/openapi-inventory.py \
module13-api-security/01-recon/openapi.json \
--output module13-api-security/01-recon/endpoints.csv

column -s, -t module13-api-security/01-recon/endpoints.csv
```

Recon gap

Machine-readable docs mungkin tidak lengkap. Module lab sengaja punya deprecated endpoint yang tidak tercantum di OpenAPI; tester tetap harus menggabungkan documentation, traffic, error behavior, source references yang memang tersedia, dan version inventory.

7. Authentication vs Authorization Layers



Empat pertanyaan berbeda yang sering tercampur

Authentication menjawab siapa caller. Authorization kemudian harus divalidasi pada object, property, dan function. Token valid tidak berarti caller boleh seluruh object atau operation yang tersedia.

8. BOLA / IDOR - Object-Level Authorization

BOLA terjadi ketika endpoint menerima object identifier dan server tidak memastikan caller memang boleh melakukan action terhadap object tersebut. Identifier sequential maupun UUID sama-sama bukan kontrol akses.

Hands-on 8.1 - Cross-user order access

```
# Alice own order
curl -s http://127.0.0.1:8130/api/v1/orders/101 \
-H 'Authorization: Bearer token-alice' | jq .

# Same Alice token, Bob's order
curl -s http://127.0.0.1:8130/api/v1/orders/102 \
-H 'Authorization: Bearer token-alice' | jq .
```

Jika kedua request berhasil, root cause bukan "ID mudah ditebak". Root cause: object-level authorization tidak divalidasi setelah object ditemukan.

Remediation

Load object lalu authorize berdasarkan authenticated principal + requested action + object policy. Jangan hanya membandingkan user_id dari token dengan arbitrary ID parameter jika model bisnis lebih kompleks.

9. BOPLA / Mass Assignment - Property-Level Authorization

API3:2023 mencakup dua arah masalah: response memaparkan property yang caller tidak perlu lihat, atau request mengizinkan caller menulis property yang seharusnya server-controlled.

Hands-on 9.1 - Observe excessive property exposure

```
curl -s http://127.0.0.1:8130/api/v1/me \
-H 'Authorization: Bearer token-alice' | jq .
```

Catat apakah response mengandung field yang tidak dibutuhkan client, misalnya internal_note. Data minimization adalah bagian API design, bukan sekadar cosmetic filtering.

Hands-on 9.2 - Mass assignment role

```
curl -s -X PATCH http://127.0.0.1:8130/api/v1/me \
-H 'Authorization: Bearer token-alice' \
-H 'Content-Type: application/json' \
-d '{"role":"admin"}' | jq .
```

Root cause

Generic object binding/patch logic menerima property role dari client. Fix: explicit writable-field allow-list per operation + authorization per property; model response juga harus explicit.

10. BFLA - Function-Level Authorization

BFLA terjadi ketika caller terautentikasi bisa memanggil administrative/sensitive operation tanpa role/function authorization yang benar.

Hands-on 10.1 - Admin operation with user token

```
curl -s http://127.0.0.1:8130/api/v1/admin/stats \
-H 'Authorization: Bearer token-bob' | jq .
```

Kalau endpoint berhasil, jangan laporkan sebagai authentication failure: token Bob valid. Yang gagal adalah authorization terhadap function administratif.

Remediation

Enforce deny-by-default authorization pada server untuk setiap function/route. Jangan mengandalkan menu/UI yang menyembunyikan admin endpoint.

11. Resource Consumption & Rate Limits

API4 bukan hanya requests-per-second. Parameter pagination, export size, GraphQL depth, file size, expensive third-party calls, OTP/SMS/email, dan batch operations dapat mengonsumsi CPU, memory, bandwidth, atau biaya.

Hands-on 11.1 - Parameter-driven resource size

```
curl -s 'http://127.0.0.1:8130/api/v1/report?limit=5' \
-H 'Authorization: Bearer token-alice' | jq '{requested,returned}'

curl -s 'http://127.0.0.1:8130/api/v1/report?limit=1500' \
-H 'Authorization: Bearer token-alice' | jq '{requested,returned}'
```

Do not stress-test

Cukup observasi behavior dengan request kecil. Lab mempunyai internal safety cap 2000. Pada production assessment, volume/rate testing harus explicit di RoE karena dapat menyebabkan DoS atau cloud cost.

Hands-on 11.2 - Low-volume rate observation

```
python labs/rate-check.py \
'http://127.0.0.1:8130/api/v1/report?limit=1' \
--token token-alice \
--count 8
```

Perhatikan status code, Retry-After, dan latency. Delapan request bukan proof bahwa endpoint aman/tidak aman terhadap DoS; ini hanya exercise melihat apakah basic throttling signal ada.

12. Sensitive Business Flows

API6 membahas functionality yang secara teknis valid tetapi berbahaya jika dieksekusi otomatis/berulang: reservasi inventory, redeem coupon, posting, signup, referral, checkout, ticket purchase, dan sejenisnya.

Hands-on 12.1 - Repeat a one-time-looking action

```
for i in 1 2 3; do
  curl -s -X POST http://127.0.0.1:8130/api/v1/coupons/redeem \
  -H 'Authorization: Bearer token-bob' \
  -H 'Content-Type: application/json' \
  -d '{"code":"WELCOME"}' | jq '{redeemed,count_for_user}'
done
```

Business question: apakah coupon seharusnya sekali per user, sekali per account, atau reusable? Security finding membutuhkan expected business invariant, bukan hanya “endpoint bisa dipanggil tiga kali”.

13. API Inventory & Version Management

API9 muncul ketika organisasi kehilangan visibility atas versi, host, environment, documentation, dan deprecated/shadow endpoints. API lama sering tertinggal dengan authorization atau logging yang lebih lemah.

Hands-on 13.1 - Deprecated endpoint discovery

```
curl -i -s http://127.0.0.1:8130/api/v0/debug/users \
-H 'Authorization: Bearer token-alice' | sed -n '1,25p'
```

Endpoint v0 tidak tercantum pada OpenAPI v1. Perhatikan header Deprecation dan data yang dikembalikan. Inventory harus mencakup route yang deployed, bukan hanya yang documented.

14. JWT & API Tokens in Context

JWT adalah format claims; API juga dapat memakai opaque token, API key, mTLS, session cookie, signed request, atau combination. Format token tidak menentukan authorization model.

JWT refresher

JWT payload dapat dibaca client. Trust baru ada setelah signature/MAC dan claim validation benar. RFC 7519 mendefinisikan JWT sebagai compact representation of claims; API tetap harus memvalidasi iss/aud/exp/nbf dan authorization setelah identity diketahui.

Local lab sengaja memakai opaque training token agar jelas bahwa BOLA/BOPLA/BFLA dapat terjadi meskipun authentication token sendiri tidak rusak.

15. GraphQL API Testing

GraphQL tetap API, tetapi attack surface berpusat pada schema, query/mutation, resolver authorization, introspection, aliases/batching, query cost, dan field-level exposure. Schema visibility bukan authorization.

Hands-on 15.1 - PortSwigger GraphQL path

1. Buka GraphQL API vulnerabilities di Web Security Academy.
2. Mulai dari lab tentang finding GraphQL endpoints / introspection.
3. Map query, variables, operationName, object types, dan mutation yang tersedia.
4. Uji object/field authorization sesuai objective lab.
5. Jika lab memakai aliases/batching, catat bagaimana request tunggal dapat menghasilkan banyak resolver execution.
6. Tulis finding dengan root cause pada resolver/policy, bukan "GraphQL enabled".

GraphQL guardrails

Jangan menjalankan deep/recursive query atau high-volume batching ke API produksi tanpa explicit authorization. Query complexity testing dapat menjadi availability test.

16. Unsafe Consumption of Third-Party APIs

API10 menyoroti trust yang terlalu tinggi terhadap integrated API. Response third-party tetap untrusted input: bisa malformed, malicious, stale, overprivileged, atau berasal dari compromised dependency/provider.

Question	What to verify
Transport	HTTPS/TLS verification; no insecure fallback
Authentication	Credential scope; rotation; no token leakage in logs
Response schema	Type/size/schema validation before use
Redirects/URLs	Destination validation; avoid blind redirects/SSRF chain
Timeout/retry	Bounded timeout, retry budget, circuit breaker
Authorization	Third-party data must not bypass local access policy

17. Black-Box Practical Challenge

Jangan buka app.py terlebih dahulu. Gunakan Burp/curl dan documentation yang memang diekspos lab.

- [] Buat endpoint inventory dan tandai documented vs discovered.
- [] Gunakan Alice dan Bob untuk menemukan minimal satu object-level authorization flaw.
- [] Temukan property yang tidak semestinya dibaca atau ditulis.
- [] Temukan function administratif yang authorization-nya lemah.
- [] Analisis satu resource/business-flow weakness tanpa high-volume testing.
- [] Temukan deprecated/shadow API version.
- [] Tulis minimal 4 findings lengkap dengan baseline, modified request, response delta, root cause, impact, remediation.

Finding helper

```
python labs/finding-note.py \
  module13-api-security/04-findings/bola-order.md \
  --title 'Broken object-level authorization on orders' \
  --category 'API1:2023 BOLA' \
  --endpoint 'GET /api/v1/orders/{id}' \
  --evidence 'Alice token can read Bob order 102.' \
  --impact 'Authenticated user can read another user order.' \
  --remediation 'Authorize requested order against caller server-side.'
```

18. Evidence & Reporting Discipline

API finding harus reproducible tanpa dump secrets berlebihan. Simpan request/response yang relevan, redact token bila report keluar dari secure workspace, dan catat identity/role yang digunakan.

Finding evidence	Minimum
Identity	Role/user used; redact real token
Baseline	Expected authorized request + response
Mutation	One changed ID/property/function/parameter
Delta	Status/body/side effect difference
Root cause	Missing object/property/function/business rule check
Impact	Data/action/business effect in scope
Remediation	Server-side control; concrete policy location

Do not paste live secrets

Authorization header, API key, refresh token, signed URL, session cookie, webhook secret, dan third-party credentials harus diperlakukan sebagai secrets. Gunakan dummy/local token pada public repo.

19. Knowledge Check

1. Apa beda authentication dengan object-level authorization?
2. Kenapa UUID tidak memperbaiki BOLA?
3. Apa dua arah utama BOPLA?
4. Kenapa hidden admin button bukan function authorization?
5. Apa beda rate limit dengan resource limit?
6. Kenapa API6 membutuhkan business context?
7. Apa risiko API version lama yang tidak terinventarisasi?
8. Kenapa OpenAPI docs bukan bukti semua deployed endpoint?
9. Kenapa JWT decode tidak sama dengan verification?
10. Apa yang harus divalidasi saat mengonsumsi third-party API?
11. Kenapa GraphQL introspection exposure tidak otomatis critical?
12. Mengapa tester butuh dua identity untuk authorization testing?
13. Kapan high-volume testing harus dihentikan/di-escalate?
14. Apa evidence minimum untuk BOLA finding?
15. Bagaimana remediation mass assignment yang production-ready?

20. Exit Criteria

- [] Bisa membangun endpoint inventory dari OpenAPI + observed traffic.
- [] Bisa membedakan BOLA, BOPLA, BFLA, dan authentication flaw.
- [] Bisa menjalankan two-user differential authorization test.
- [] Bisa menguji writable/readable properties secara minimal dan terkontrol.

- [] Bisa mengobservasi resource/rate behavior tanpa stress test.
- [] Bisa menjelaskan business-flow vulnerability dengan expected invariant.
- [] Bisa menemukan dan menilai deprecated API version.
- [] Bisa menjelaskan JWT sebagai token format, bukan authorization control.
- [] Bisa mengikuti GraphQL Academy lab dan memetakan schema/resolver authorization.
- [] Bisa menulis finding API yang reproducible dan tidak membocorkan secrets.

21. Cheat Sheet

```
# Login local identities
curl -s -X POST http://127.0.0.1:8130/api/v1/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"alice","password":"training123"}' | jq .

# Authenticated GET
curl -s http://127.0.0.1:8130/api/v1/me \
  -H 'Authorization: Bearer token-alice' | jq .

# BOLA differential
curl -s http://127.0.0.1:8130/api/v1/orders/101 -H 'Authorization: Bearer token-alice' | jq .
curl -s http://127.0.0.1:8130/api/v1/orders/102 -H 'Authorization: Bearer token-alice' | jq .

# Property-level write
curl -s -X PATCH http://127.0.0.1:8130/api/v1/me \
  -H 'Authorization: Bearer token-alice' \
  -H 'Content-Type: application/json' \
  -d '{"role":"admin"}' | jq .

# Cleanup
./labs/api-lab.sh cleanup
```

22. References

OWASP API Security Project - Project overview and API Security Top 10

<https://owasp.org/projects/api-security-project>

OWASP API Security Top 10 - 2023 - Current Top 10 edition used by this module

<https://api-security.owasp.org/editions/2023/en/0x11-t10/>

OWASP API1:2023 - Broken Object Level Authorization - BOLA model and remediation guidance

<https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/>

OWASP API3:2023 - Broken Object Property Level Authorization - Property exposure and mass assignment root cause

<https://api-security.owasp.org/editions/2023/en/0xa3-broken-object-property-level-authorization/>

OWASP API4:2023 - Unrestricted Resource Consumption - Resource/operational cost perspective

<https://api-security.owasp.org/editions/2023/en/0xa4-unrestricted-resource-consumption/>

OWASP API10:2023 - Unsafe Consumption of APIs - Third-party API trust

<https://api-security.owasp.org/editions/2023/en/0xaa-unsafe-consumption-of-apis/>

PortSwigger Web Security Academy - API Testing - API reconnaissance and testing labs

<https://portswigger.net/web-security/learning-paths/api-testing>

PortSwigger Web Security Academy - GraphQL - GraphQL endpoint/schema/security labs

<https://portswigger.net/web-security/graphql>

RFC 7519 - JSON Web Token - JWT standard

<https://www.rfc-editor.org/rfc/rfc7519.html>

Next module

Module 14 - Exploitation Fundamentals: exploit research, CVE/PoC triage, safe public-exploit review, shells/payload concepts, Metasploit fundamentals, file transfer, evidence, and deliberately vulnerable training machines.